SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1004	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Akshaya Sekar	**Reg. No.:**	1911260204
Date:	10.10.2026	**Duration:**	60 minutes

Code of Conduct

I Akshaya Sekar/1911260204 *(Name/RegNo)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Akshaya Sekar

Case Study Analysis: Intelligent Hotel Management and Guest Experience System

1. Understanding the Problem Statement

Hotels handle room reservations, guest records, check-in, housekeeping, billing and service requests. When these run on separate tools or paper registers, information is not shared, which causes double bookings, slow responses and billing mistakes. The task is to design one integrated system that connects all these activities and gives guests faster, more reliable service.

Objectives: manage room inventory, process reservations, handle check-in and check-out, track housekeeping, manage guest requests, generate accurate bills, maintain guest histories and produce performance analytics.

Scope: the system covers the front office, housekeeping, billing, guest services and management reporting for a single hotel or chain. Payroll, supplier procurement and full accounting are outside its scope.

2. Key Stakeholders and Challenges

- **Guests:** book rooms, request services and pay bills.
- **Front desk staff:** handle reservations, check-in/out and allocation.
- **Housekeeping and maintenance:** clean rooms, report faults and update room status.
- **Hotel management:** monitor occupancy, revenue and service quality.
- **Accounts staff and payment gateways:** verify invoices and process payments.

Each stakeholder depends on the others: a late check-out affects housekeeping, which affects the next arrival, which affects the front desk and the guest. The system must therefore serve all of them from the same data.

Challenges: guests face booking conflicts and long waits; front desk staff work with outdated room availability; housekeeping gets late or unclear cleaning instructions; managers lack timely data for decisions; and accounts staff correct

billing errors manually. Together these problems reduce efficiency, increase costs and lower guest satisfaction.

3. Analysis of the Current Approach

Many hotels use separate tools for booking, a manual register or spreadsheet for housekeeping, standalone billing software and phone calls for guest requests. Its strengths are low cost, familiarity to staff and little training. Its limitations are far greater: data is duplicated and inconsistent, room status is not updated in real time, requests are lost or delayed, bills are error-prone, guest history is not retained and reports must be compiled by hand. The main gap is the lack of a single, real-time source of truth shared by all departments.

Other gaps include no automatic link between check-out and room cleaning, no tracking of request response times, and no analytics on guest behaviour. These gaps directly produce the delays and conflicts described in the problem statement.

4. Proposed Software Solution

The proposed solution is a web and mobile application with a central database and role-based access for guests and staff. Its major modules are:

- **Room Inventory:** room types, rates and live availability.
- **Reservation:** online and walk-in booking with conflict prevention.
- **Check-in/Check-out:** identity capture, room allocation and key issue.
- **Housekeeping:** automatic task assignment and room status tracking.
- **Guest Requests:** ticketing for services with priority and tracking.
- **Billing:** automatic charges, taxes, invoices and online payment.
- **Guest Profile and Analytics:** history, preferences, occupancy and revenue dashboards.

Functional requirements: users can search and book rooms, staff can allocate rooms and update status, housekeeping receives tasks, guests can raise and track requests, the system generates invoices, and managers can view reports. Guest profiles are stored so that returning guests receive personalised service.

Each module exposes its data through well-defined interfaces, so a change in billing rules or room categories does not affect reservations or housekeeping. A single relational database stores rooms, bookings, guests, tasks, requests and invoices, and role-based dashboards show each user only what they need.

The system follows a three-tier architecture: a presentation layer (web and mobile screens), a business logic layer (booking rules, allocation, billing calculation) and a data layer. Notifications by email or SMS confirm bookings, alert housekeeping and remind guests of payments.

Non-functional requirements: response time under three seconds, 99.5% availability, data encryption, support for concurrent users, a simple interface and daily backups.

Software engineering principles: Modularity is achieved by separate, loosely coupled modules with defined interfaces. Scalability comes from a layered, cloud-deployable architecture that can add rooms or properties. Maintainability is supported by clean code, documentation and version control. Reliability uses transaction control to prevent double booking, plus backups. Usability is provided through role-specific, easy screens. Security uses authentication, role-based access, encrypted payments and audit logs.

5. Chosen Methodology: Agile (Scrum)

Scrum is selected because hotel requirements are likely to change as staff and guests give feedback, and the system has clearly separable modules. Development is divided into two-to-three-week sprints, with reservation and room inventory delivered first, followed by check-in, housekeeping, billing and analytics. Each sprint ends with a working increment that hotel staff can review, so errors are found early and the product matches real needs. Waterfall would be too rigid for changing requirements, and Scrum's regular reviews, retrospectives and automated testing also make later maintenance and enhancement easier.

The product owner (hotel manager), Scrum master and development team work together through sprint planning, daily stand-ups and sprint reviews. A prioritised

product backlog keeps the most valuable features first, and continuous integration with automated tests keeps the system reliable after every change.

For maintenance, the backlog continues after release, so new requests such as a spa booking module or a seasonal pricing rule can be added in later sprints without redesigning the system. Regular retrospectives help the team improve its process, and user stories written in the guest's or staff member's words keep development focused on real needs.

6. Expected Outcomes and Limitations

Benefits: improved hotel operations, fewer reservation conflicts, faster guest service, better housekeeping coordination, accurate billing and higher guest satisfaction. Central data and real-time room status address each challenge identified earlier, and analytics supports better pricing and staffing decisions. Housekeeping and front desk staff share live room status, so rooms are ready sooner and waiting time at check-in is reduced.

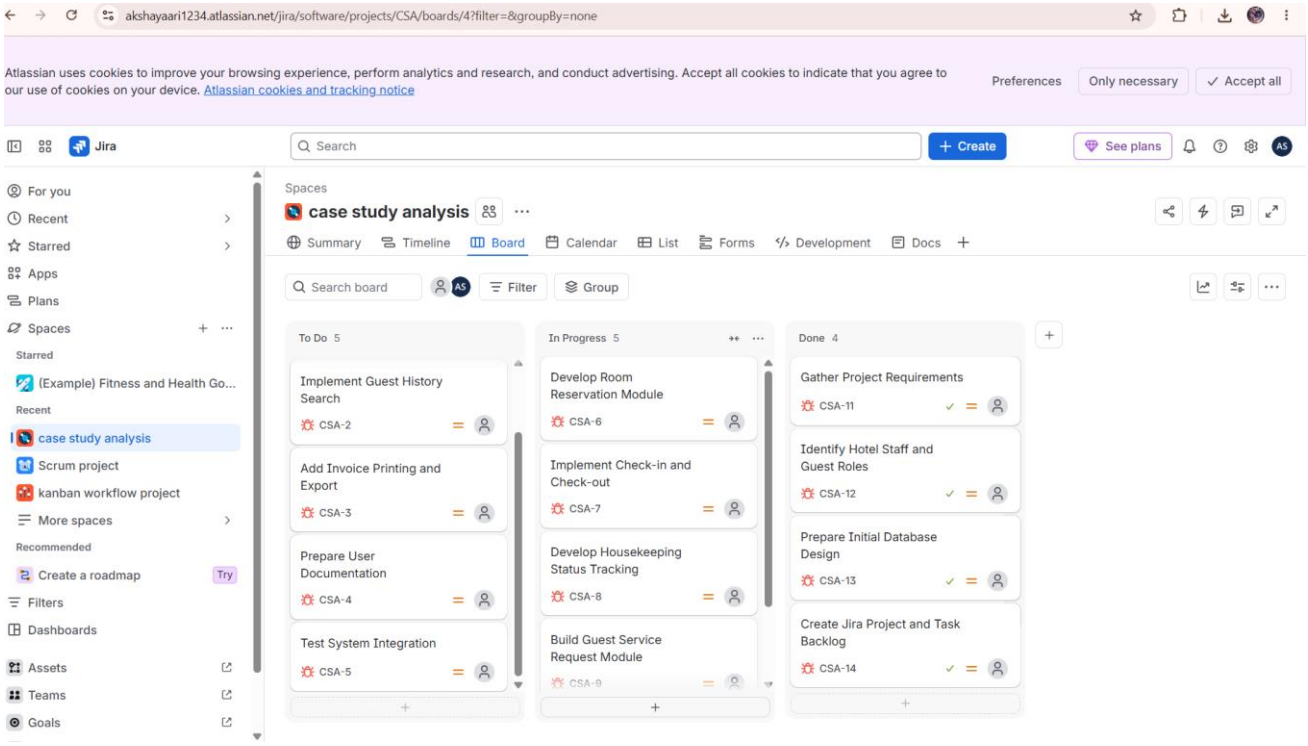
Specifically, live inventory and transactional booking address reservation conflicts, automated task assignment addresses housekeeping delays, ticketing addresses lost guest requests, and automatic invoicing addresses billing errors. Success can be measured through lower booking errors, shorter check-in time, faster request resolution and higher guest ratings.

Risks and limitations: staff resistance to change, migration of old data, internet or server downtime, data privacy obligations and integration with existing payment or door-lock systems. Initial development and training also require investment. These risks can be reduced through phased roll-out, staff training, a backup server and compliance with data-protection rules.

Future enhancements: AI-based demand forecasting and dynamic pricing, chatbot or voice assistants for guests, mobile keys, IoT-based smart rooms, multilingual support and integration with online travel agencies.

Conclusion

The Intelligent Hotel Management and Guest Experience System replaces disconnected tools with one integrated platform. By applying modular design, strong security and an Agile Scrum process, the hotel can reduce booking conflicts, coordinate housekeeping better, bill accurately and respond to guests quickly. The system is therefore a practical, scalable and maintainable solution that improves both hotel operations and the guest experience.



Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25